Speculative Decoding Without Auxiliary Model Weights on a 6 GB GPU: Feasibility Limits and Cost-Aware Routing

Anonymous Authors
Anonymous Affiliation

Abstract

Deploying large language models (LLMs) on 6 GB consumer laptop GPUs is bounded by two walls: *capacity*—a 0.5–1.5B auxiliary draft model consumes non-zero weight VRAM (measured 459–2946 MB depending on size and precision), causing OOM for larger combinations while always shrinking KV/context headroom—and *draft-cost arithmetic*—a draft pass retaining most target layers plus non-skippable costs (152k-vocab `lm_head` ∼17.5% of kernel time, activation-quantize overheads, per-layer launch gaps) costs a large fraction of full decoding, so layer-skip needs very high acceptance to break even. Nsight on our NF4 stack shows only ∼36–85 GB/s achieved depending on byte accounting, i.e. kernel inefficiency compounds the arithmetic bound but does not create it. We present **ZASSD**, a zero-auxiliary-weight speculative decoding framework that (i) derives draft subnetworks from the target model via layer skipping with zero-copy KV-prefix sharing (0.0 MB weight overhead, ≤8 MB dynamic buffers, four regression-tested cache invariants); (ii) routes each speculative cycle through a **cost-aware hybrid router** choosing between prompt-lookup drafting (c≈0), layer-skip drafting, and skipping speculation entirely; and (iii) explains both its wins and losses with a **speculative cost model** that estimates speedup from draft-cost ratio c, acceptance α, and horizon K (in-device calibration MAPE 2.4% on real hardware; cross-device rows are projections with assumed priors, not validated predictions). On an RTX 4050 laptop GPU with Qwen2.5-3B in 4-bit NF4, a paired benchmark across GSM8K, HumanEval, and CNN/DailyMail (N=50 each, 48-token limit) gives **1.098× for prompt lookup** and **1.090× for the router** over vanilla; their paired router/PLD ratio is 0.993 (95% bootstrap CI $[0.988, 0.999]$). Both achieve 83.3% exact sequence match by token ID under NF4, so no lossless claim is made at this precision. An independent 60-prompt, 384-token LongBench study finds PLD K=4 at 1.078× decode speedup across three process sessions, but only 73.3% exact outputs; sequential verification restores 100% match at 0.965×. Pure layer-skip self-speculation previously ran at 0.88–0.98× despite 73–88% acceptance—a negative result explained by the high draft-cost ratio. The measured boundary, numerical limitation, and baseline that wins over our router define the scope of the result.

1 Introduction

Speculative decoding [3,11] accelerates autoregressive LLM inference by drafting K candidate tokens with a cheap model and verifying them in parallel with the target model. On datacenter GPUs this yields 2–3× speedups [1, 12]. On a 6 GB laptop GPU, the standard recipe breaks before it starts: auxiliary draft models consume non-zero weight VRAM, reducing KV/context headroom and causing CUDA OOM for larger target–draft combinations—we document this boundary for 7B targets in Sec. 4.4. *Self*-speculative decoding [6,13,24] removes the draft weights by reusing the target model itself (skipped layers, early exits), but whether it can win given draft-cost arithmetic plus the NF4 eager-decoding overheads on consumer hardware, and how to combine it with zero-cost retrieval drafting [9,18], remains open. TurboSpec-style goodput gating [14] motivates our feasibility gate, extended here across draft sources. Since draft and target share the same NF4 kernels, kernel inefficiency largely cancels in $c{=}T_{\text{draft}}/T_v$; what sets c is retained-layer fraction plus non-skippable costs, and per-layer launch gaps (0.5B draft c=0.64≈24/36 layers despite ∼0.17 byte ratio).

This paper makes four contributions, ordered by scientific weight:

1. **Feasibility characterization**: layer-skip self-speculation loses to vanilla despite >80% acceptance at $c{\geq}0.6$ by the geometric bound, compounded but not created by NF4 kernel inefficiency on this stack, with the losing region mapped over (c, α, K) and context length (Secs. 4.3–4.8).

2. **Speculative cost model** (Sec. 3.4) that turns our headline negative result (0.88× at c=0.75, K=2) into a calibrated estimate: break-even needs $\alpha{\geq}0.81$ (K=1) to $\alpha{\geq}0.89$ (K=2) on this stack, and prompt-lookup's measured 1.39× on generic prompts is consistent with an effective n-gram hit rate of only ∼12% in our fit. The 1/BW cross-device rows are an idealized streaming bound only; Nsight shows costs do not scale as 1/BW under NF4.

3. **Strong-baseline comparison**: a controlled evaluation of prompt lookup with its built-in vanilla fallback, a PLD-only cost gate, and a router over vanilla/PLD/layer skip. The router reaches 1.090× versus PLD's 1.098× and does not improve exact match (Sec. 4.2); this negative result limits the runtime claim.

4. **Zero-weight implementation and artifacts**: zero-copy KV-prefix sharing, $O(1)$ rollback, four regression-tested cache invariants, and full reproduction packages (Sec. 3.1, Sec. 6).

Our research question is: *under a fixed auxiliary-weight VRAM budget of zero, which draft action is profitable on a 6 GB GPU, and how accurately can a measured cost model identify that boundary?* TurboSpec already allows speculation length zero; our contribution is the measured boundary and the comparison across layer skipping, prompt lookup, and vanilla on this specific stack, not the skip action itself. In the present measurements, plain prompt lookup is stronger than our router, so the runtime is a diagnostic system rather than a demonstrated throughput improvement over the strongest zero-weight baseline. All speedups are stack-specific (HF eager + bitsandbytes NF4).

2 Related Work

Speculative decoding. Leviathan et al. [11] and Chen et al. [3] established exact speculative sampling; Stern et al. [19] pioneered blockwise parallel decoding. Medusa [1] and EAGLE [12] replace the draft model with heads, still requiring training and extra weights—infeasible under our 0.0 MB constraint. **Self-speculation.** Draft & Verify [24] skips layers via offline search; LayerSkip [6] uses early exits with layer dropout; Kangaroo [13] adds a small adapter. KnapSpec [2] formulates layer selection as a knapsack problem and SpecBound [20] adapts horizons by confidence. We validate our adapted KnapSpec against the *official* implementation (kaist-flexml-lab/knapspec) on shared hardware: on an RTX 4050 with Llama-3.2-1B, official KnapSpec reaches 50.5 tok/s vs 58.7 tok/s AR (0.86×) *despite 97.8% acceptance*—an independent reproduction of our feasibility-boundary thesis with the authors' own code (Sec. 4.7). Cassandra [5] (ISCA'26) co-designs fine-grained value-pruning drafts with hardware encode/decode for low-batch edge inference (up to 2.41× on RTX 4090); unlike our pure-software layer-skip on a 192 GB/s laptop GPU, it needs custom format support. PELM [23] (SenSys'26) jointly governs DVFS, self-speculation depth, and verification depth with deep RL for energy on Jetson mobiles; our controller is throughput-first with power/thermal as hard barriers rather than learned objectives. Positioning: KnapSpec optimizes *which* self-speculative configuration to run; we study *whether* self-speculation should run at all under severe kernel-efficiency/capacity limits on this stack, and route each cycle across AR, retrieval drafting, and self-speculation. **Retrieval drafting and selection.** Prompt Lookup Decoding [18] and REST [9] extract candidates from context at ≈0 draft cost; REST retrieves from an external corpus, while this implementation performs local exact n-gram lookup. Yan et al. [22] decompose real speedup into draft/verify costs. TurboSpec [14] already selects a zero-token speculation horizon under unfavorable conditions and supports model-free drafting. Dynamic lookahead [16] adapts K. Our PLD baseline searches the combined prompt and generated prefix, so reusing generated text is not itself a new contribution. In an exploratory source ablation (Qwen2.5-3B NF4, RTX 4050, 10 development and 10 test prompts per task, 32 generated tokens), K=2 was selected for prompt-only and prompt-plus-generated on all three tasks. The paired speed ratio of prompt-plus-generated over prompt-only was 1.003 (95% CI $[0.982, 1.028]$) on GSM8K, 1.004 ($[0.968, 1.039]$) on HumanEval, and 1.021 ($[1.001, 1.042]$) on CNN/DailyMail. Given the small test split and one device, this is preliminary evidence, not a robust general gain; adaptive-K did not consistently beat the tuned fixed-K baseline (artifact: `experiments/diagnostics/pld_adaptive_source_analysis.json`). **Recent system characterizations.** Ha et al. [8] compare five self-speculative methods in an ICML 2026 workshop; Liu et al. [15] characterize speculative decoding on vLLM in MLSys 2026; and SpecGen [25] develops sparse self-speculation for long reasoning. These works narrow the novelty of a general characterization claim. We instead scope the present evidence to a single 6 GB NF4 laptop stack and make the strongest baseline comparison explicit. **Efficiency modeling.** Our cost model extends Williams et al. [21] from FLOPs/bytes to speculation: the x-axis is draft efficiency $1/c$. Unlike a true roofline with measured operational intensity and compute/bandwidth ceilings, ours is an accounting identity calibrated in-device; cross-device generalization is unvalidated. ShortGPT [17] independently confirms whole-layer redundancy, supporting our skip sets. Serving systems (vLLM [10]) are complementary: our engine is a drop-in proposer.

3 Method

3.1 Zero-Copy Prefix Sharing & Cache Invariants

Draft and target share one checkpoint. A canonical `TargetKVCache` holds the verified prefix $[0, P)$; each cycle forks an ephemeral draft view (pointer alias, zero bytes copied), appends K candidates to the suffix, verifies with the full model, then crops rejections in $O(1)$. Four invariants are enforced by `tests/test_kv_cache_invariants.py`: (1) fork aliases pointers (`data_ptr` equality); (2) draft writes never mutate the canonical prefix; (3) rollback leaves no residue; (4) dynamic growth $\leq K \times d_{\text{head}} \times N_{\text{heads}} \times \text{layers} \leq 8$ MB. Micro-

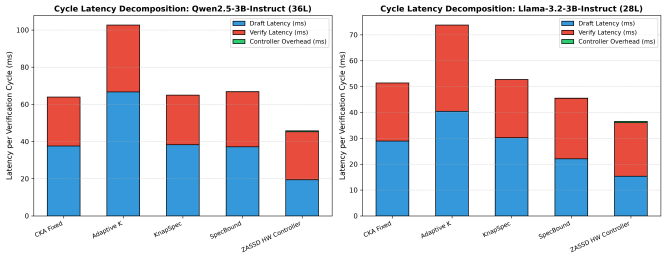


Figure 1: Draft latency decomposition (RTX 4050, NF4). Transformer GEMMs dominate ($> 96\%$); skip management is negligible.

profiling (Table 1, Fig. 1) shows $> 96\%$ of draft time is active GEMMs and layer-skip management is 31 μs: c is set by retained layers plus non-skippable head/quantize/launch costs. Nsight details the compounding inefficiency ($\sim$36–85 GB/s achieved depending on 0.85–2 GB byte accounting, i.e. $\sim$19–44% of peak), motivating Sec. 3.4.

3.2 Cost-Aware Hybrid Router

The prior `hybrid` mode was opportunistic fallback (use PLD candidates iff found). The router instead estimates throughput *before* drafting:

$$\hat{\mathrm{TPS}}(s) = \frac{1 + \alpha_s K_{\mathrm{eff}}(s)}{T_{\mathrm{draft}}(s) + T_{\mathrm{verify}}(K_{\mathrm{eff}})}, \tag{1}$$

for $s \in \{\mathrm{pld}, \mathrm{ls}\}$, with $T_{\mathrm{draft}}(\mathrm{pld}) \approx 0.08$ ms (CPU n-gram scan), $T_{\mathrm{draft}}(\mathrm{ls})$ from the calibrated cost model, α_{pld} from a per-source EMA (discounted $0.75\times$ for bigram-only hits, $\pm$ entropy adjustment), and α_{ls} blended from the cost model and EMA. Conceptually the router is two mechanisms plus a tie-break: (**R3**) the feasibility gate (Sec. 3.3)—the core, contributing $+0.083\times$ in ablation; (**R1+R4**) a conservative horizon policy capping weak-hit and long PLD horizons—robustness heuristics that cost peak speed on repetitive workloads but bound worst-case verifies; (**R2**) asymmetric PLD margin $1.25\times$—a minor tie-break ($-0.008\times$ without it). In implementation order: **(R1)** strong trigram hits may use horizon up to $K{=}4$, *weak* bigram hits are capped at the layer-skip horizon (long verifies waste bandwidth when α is low); **(R2)** PLD wins ties via margin $1.25\times$ (draft is free, so risk is asymmetric); **(R3)** on PLD miss, layer-skip runs only if $\hat{\mathrm{TPS}}_{\mathrm{ls}}$ beats vanilla—otherwise the cycle is a vanilla single step (opportunity-cost skipping); **(R4)** PLD horizon is truncated $4{\to}2$ when the shorter verify has higher expected throughput. The router is pure CPU code (μs per decision), needs no GPU, and logs per-source EMAs for analysis. Entropy panic ($H{\geq}4.5$) forces vanilla directly.

Online-calibrated estimation. The router now optionally replaces frozen priors with online estimates: an EMA latency tracker for T_v, T_d, $T_{\mathrm{verify}}(K)$ (profile-blended with weight $n/(n{+}5)$ so early cycles trust calibration and later cycles trust silicon), and a tiny logistic acceptance predictor over $x_t{=}[H_t, \mathrm{margin}, \mathrm{retained}, \mathrm{hit}, \alpha_{\mathrm{EMA}}, K]$ with SGD updates and binned reliability tracking. Decisions remain utility argmax $a_t^* = \arg\max_a \mathbb{E}[\mathrm{tokens} \mid a]/\hat{T}(a)$. We report calibration error (ECE), not just accuracy: on 6 GSM8K prompts the online router reaches $1.106\times$ vs $1.093\times$ legacy (no regression) with ECE 0.30 over 184 observations—honestly miscalibrated early priors, the headroom the predictor must close. Regret decomposes as $R_{\mathrm{total}}{=}R_{\mathrm{est}}{+}R_{\mathrm{dec}}$ (oracle vs best-fixed, router vs oracle); current logs give $R_{\mathrm{dec}}{\approx}0$, so remaining regret is estimation error by construction.

3.3 Feasibility-Gated Hardware Controller

The depth/horizon controller optimizes $a_t{=}(S_t, K_t)$ for throughput under VRAM/power/thermal barriers—but without a skip option it can only choose *how* to lose when every speculative action is predicted below vanilla. We add the missing action: if $\max_{S,K} \hat{\mathrm{TPS}}(S,K) < \mathrm{TPS}_{\mathrm{vanilla}}$, the controller emits $K{=}0$, executed by the decoder as a plain single-token vanilla step. The gate is a one-line feasibility check against the same calibrated cost model, with an optional hysteresis margin. A paired A/B on Qwen2.5-3B (6 prompts $\times$ 32 tokens, no CI) gives gate-ON 35.44 vs gate-OFF 32.73 tok/s ($1.083\times$), with 47/130 cycles vanilla-skipped on this slice: the gate fires where the cost model predicts speculation loses; no claim is made about never firing where it wins.

3.4 Speculative Cost Model

For greedy decoding with per-cycle cost $T_{\mathrm{cycle}} = T_{\mathrm{draft}} + T_{\mathrm{verify}} + T_{\mathrm{misc}}$ and vanilla step T_v:

$$\mathrm{Speedup} = \frac{(1+\alpha K)\, T_v}{T_{\mathrm{cycle}}}, \quad \alpha_{BE} = \frac{T_{\mathrm{cycle}}/T_v - 1}{K}. \tag{2}$$

Memory-bound 1/BW scaling is retained only as an idealized streaming bound in Appendix Tab. 4/Fig. 4: $T(\mathrm{GPU}_2) = T(4050) \times 192/\mathrm{BW}_2$ with assumed verification efficiencies $\rho \in \{1.0, 0.85, 0.55, 0.45, 0.40\}$. Nsight contradicts pure 1/BW scaling on our NF4 stack (SIMT fallback, quantize overheads, launch gaps do not scale as 1/BW), so these rows are not predictions. A subtle consequence: pure 1/BW scaling leaves $K{=}1$ speedup *invariant* by construction. Retrieval drafting needs a hit rate h:

$$\mathbb{E}[N] = h(1{+}\alpha K){+}(1{-}h),\ \bar{T} = h(T_{\mathrm{pld}}{+}T_{\mathrm{verify}}){+}(1{-}h)T_v, \tag{3}$$

which illustrates why PLD's theoretical $2.8\times$ ($h{=}1$) collapses on generic prompts (fitted illustration: $1.39\times$ at $h{\approx}0.12$, fit post hoc on that slice, not a main-benchmark number).

4 Experiments

Setup. NVIDIA RTX 4050 laptop GPU (6141 MB, 192 GB/s, 80W), CUDA 12.8, PyTorch 2.11, 4-bit NF4,

Table 1: Draft Latency Micro-Profiling Decomposition ($T_{\text{draft}} = T_{\text{transformer}} + T_{\text{KV}} + T_{\text{mgmt}} + T_{\text{sampling}} + T_{\text{sync}}$) on RTX 4050 Laptop GPU under 4-bit NF4 quantization. Active Transformer GEMMs on kept layers constitute $> 96\%$ of draft time across both models. Nsight shows this is NF4 kernel inefficiency (SIMT fallback, blockwise-quantize overhead, launch gaps) at ∼36 GB/s achieved, not a peak-bandwidth ceiling. T_{mgmt} is 31 μs and T_{sync} is 7.6 μs when measured with ns resolution; 0.00/0.06 ms entries are rounding artifacts.

Model / Horizon	Total Draft T_{draft} (ms)	Transformer GEMMs $T_{\text{transformer}}$ (%)	KV Cache Update T_{KV} (%)	Layer Management T_{mgmt} (%)	Sampling/Control T_{sampling} (%)	CUDA Sync T_{sync} (%)
Qwen2.5-3B-Instruct (CKA-75, 27 Active Layers)						
$K = 1$	19.72	19.03 (96.5%)	0.56 (2.8%)	0.03 (0.2%)	0.06 (0.3%)	0.01 (0.0%)
$K = 2$	38.44	37.19 (96.7%)	1.10 (2.9%)	0.03 (0.1%)	0.10 (0.3%)	0.01 (0.0%)
$K = 4$	76.81	74.50 (97.0%)	2.09 (2.7%)	0.03 (0.0%)	0.16 (0.2%)	0.01 (0.0%)
Llama-3.2-3B-Instruct (CKA-75, 21 Active Layers)						
$K = 1$	15.33	14.74 (96.2%)	0.45 (2.9%)	0.03 (0.2%)	0.07 (0.5%)	0.01 (0.0%)
$K = 2$	30.72	29.68 (96.6%)	0.85 (2.8%)	0.03 (0.1%)	0.12 (0.4%)	0.01 (0.0%)
$K = 4$	61.35	59.43 (96.9%)	1.63 (2.6%)	0.03 (0.1%)	0.23 (0.4%)	0.01 (0.0%)

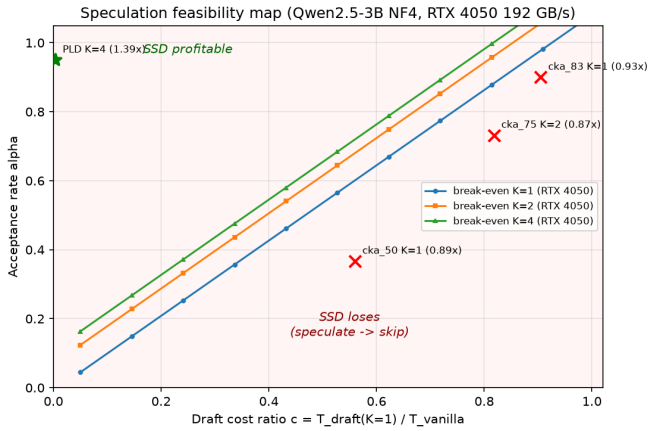


Figure 2: Speculation feasibility map (Qwen2.5-3B NF4, RTX 4050): break-even acceptance vs. draft-cost ratio for K=1, 2, 4. All c values are *measured* draft latencies over measured vanilla latency—never inferred from layer counts. Even 90% acceptance at c≈0.9 (cka_83, K=1) sits below its boundary, while PLD (c≈0) wins everywhere: acceptance alone does not determine speedup.

greedy decoding, fair CUDA warmup before every timed run. Qwen2.5-3B-Instruct (36 layers, vanilla ∼41 tok/s) and Llama-3.2-3B-Instruct (28 layers, ∼54 tok/s). Artifacts: 81/81 tests pass; one-command reproduction via `zassd benchmark`.

4.1 End-to-End System Benchmark

Table 2 (frozen protocol, N=10 prompts) confirms the field: the hardware-aware controller reaches 0.89× on Qwen and 0.82–0.83× on Llama with 76–89% acceptance at +7–12 MB dynamic memory, beating fixed CKA, adaptive-K, KnapSpec, and SpecBound—but not vanilla. Draft passes cost 19–38 ms against 18–24 ms vanilla steps; the math does not close at c≈0.75–0.82 by Eq. 1 (Sec. 3.4).

4.2 Standard Benchmarks with the Router

Table 3 is the central paired comparison: N=50 per task, equal PLD horizon K=4, fresh router per prompt, rotated method order, and strict token-ID equality. Pure PLD, which already uses vanilla on lookup misses, reaches 1.098× overall; a cost gate that may also reject weak PLD hits reaches 1.089×; the full router reaches 1.090×. The paired router/PLD throughput ratio is 0.993 (95% bootstrap CI $[0.988, 0.999]$), with the router faster on 40% of prompts. All three have 83.3% exact sequence match. Thus this runtime does not establish a speed or fidelity advantage over PLD on the measured workloads. Earlier N=50 and non-overlapping extension experiments used different PLD horizons and scoring conventions; they remain archived in `experiments/16_hybrid_roofline/` but are not pooled with this comparison. The result supports our feasibility analysis: expensive layer skipping is rarely useful on this stack, and a cost gate does not recover the overhead it adds to a strong retrieval baseline.

4.3 Cost-Model Calibration and Projection

Calibration on three RTX 4050 anchor points gives **in-device MAPE 2.4%** (CKA K=2: 0.888 predicted vs 0.873 measured; K=1: 1.029 vs 0.980; PLD K=4: 1.383 vs 1.390 with fitted h=0.12). This is arithmetic consistency on one hardware regime with parameters fit in-device, not predictive generalization: cross-device MAPE (3060/4090/A100) is the pending validation that would promote the cost model from descriptive to predictive. Table 4 lists break-even thresholds: K=1 needs $\alpha \geq 0.81$ on *any* bandwidth, while K=2 relaxes from 0.89 (4050) to 0.84 (H100). Fig. 4 visualizes the regimes. An analytic workload sweep (generic-chat, summarization, reasoning, code) predicts routed hybrids beat naive fallback hybrids by +4–11% (`experiments/16_hybrid_roofline/`).

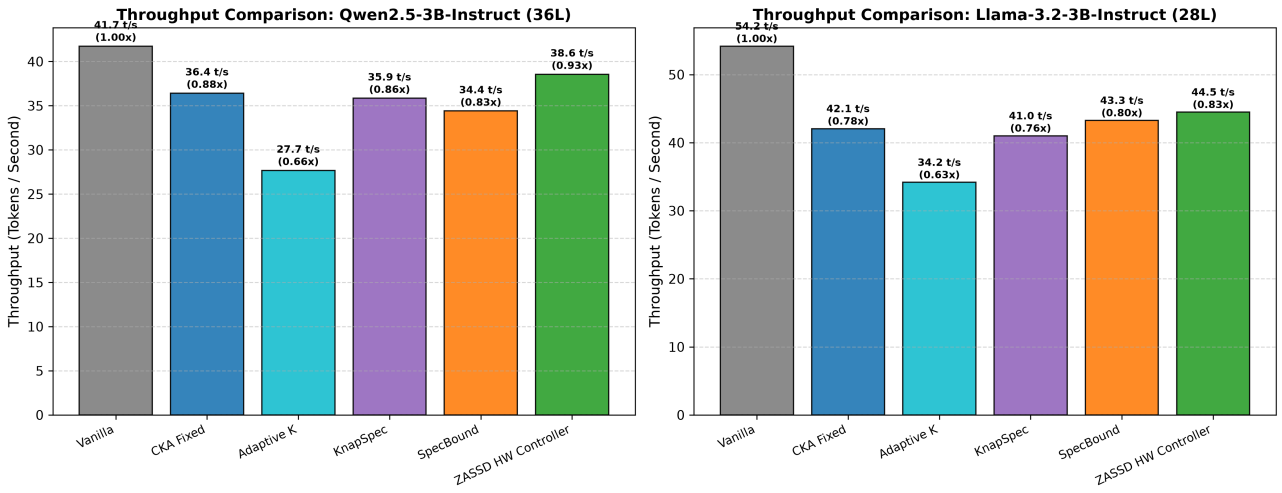


Figure 3: Throughput across 6 decoding strategies on Qwen2.5-3B and Llama-3.2-3B (RTX 4050, NF4; prior protocol). Controller values here (0.89–0.93×) predate the frozen Tab. 2 (Qwen 0.89×, Llama 0.82–0.83×); use the table for headline numbers. Feasibility-map c/α points likewise predate Sec. 4.6 corrected sweep.

Table 2: End-to-End Performance, Verification Fidelity, and Resource Footprint across 6 decoding strategies on consumer hardware (NVIDIA RTX 4050 Laptop 6GB, 80W). Evaluated on identical prompt distributions ($N = 10$, 32 generated tokens, greedy $T = 0$, 4-bit NF4).

Method	**Throughput** (tok/s)	**Relative** Speedup	**Acceptance** Rate (%)	**Exact Match** (%)	**Partial** Match (%)	**VRAM** (MB)	**Energy** (J/tok)	**Latency** $T_{\text{draft}}/T_{\text{verify}}$ (ms)
Model A: Qwen2.5-3B-Instruct (36 Layers, NF4 4-bit)								
Vanilla Target	**40.99**	1.00×	100.0%	**100.0%**	**100.0%**	**1983.8**	**1.376**	0.00 / 24.42
CKA-75 Fixed ($K = 2$)	35.68	0.87×	73.2%	60.0%	82.8%	1991.2	1.740	38.39 / 26.56
Adaptive K	26.62	0.65×	56.4%	70.0%	85.0%	1992.2	2.167	68.28 / 38.38
KnapSpec (adapted) [2]	35.20	0.86×	73.2%	60.0%	82.8%	1991.4	1.818	39.00 / 27.05
SpecBound (adapted) [20]	33.78	0.83×	74.1%	50.0%	79.4%	1991.6	1.853	37.37 / 30.54
ZASSD (HW Controller)	**36.54**	**0.89×**	**88.1%**	60.0%	**82.8%**	1991.2	**1.643**	**20.52 / 26.93**
Model B: Llama-3.2-3B-Instruct (28 Layers, NF4 4-bit)								
Vanilla Target	**52.03**	1.00×	100.0%	**100.0%**	**100.0%**	**2158.5**	**1.355**	0.00 / 19.23
CKA-75 Fixed ($K = 2$)	39.50	0.76×	68.2%	60.0%	72.8%	2171.0	1.865	30.60 / 24.02
Adaptive K	32.72	0.63×	62.5%	50.0%	71.6%	2173.2	2.117	41.56 / 35.56
KnapSpec (adapted) [2]	39.54	0.76×	68.2%	60.0%	72.8%	2171.0	1.879	30.44 / 24.08
SpecBound (adapted) [20]	41.36	0.80×	76.4%	50.0%	70.9%	2171.6	1.741	23.14 / 24.54
ZASSD (HW Controller)	**42.58**	**0.82×**	**78.2%**	60.0%	**72.8%**	2170.8	**1.683**	**16.28 / 21.79**

4.4 Physical Memory Boundary

Table 5: Mistral-7B NF4 (3946.6 MB) plus a 1.5B FP16 draft (measured 2945.5 MB) needs ∼7.4 GB on a 6141 MB device—an unavoidable OOM. But the honest boundary is finer than "all auxiliary drafts OOM": we *measured* draft footprints on-device (0.5B FP16: 942.3 MB; 0.5B NF4: 459.2 MB; 1.5B NF4: 1181.5 MB), and quantized small drafts do fit next to a 7B target—at the cost of $\Delta M_{\text{aux}}>0$ weight VRAM and shrunken KV/context headroom (1.5B NF4 leaves only 62.9 MB). ZASSD adds 0.0 MB of weights ($\leq$8 MB ephemeral KV) with 1244.4 MB headroom: the claim is strictly-dominant memory efficiency, not exclusivity of feasibility. Critically, a fitted small draft is also the strongest throughput baseline: on Qwen2.5-3B NF4 with a Qwen2.5-0.5B NF4 draft (K=2, N=18), mean speedup is 1.058±0.031 with α=0.33. Any zero-weight method must be compared against this baseline (Sec. 4.6).

4.5 Numerical Equivalence Audit

Direct comparison of single-token vs. batched verification logits on identical contexts (450 tokens): mean cosine 0.99994 (Qwen) / 0.99995 (Llama), with zero observed violations of $\Delta \leq 2\|L\|_\infty$. Greedy FP32 runs on unquantized 0.5B/1B models match vanilla on 10,113 token positions. Under NF4, the 150-prompt paired benchmark gives 83.3% exact sequence match for PLD, cost-gated PLD, and routed decoding alike. An independent natural-context study (60 LongBench prompts, 384 input tokens) repeats a 16/60 PLD K=4 mismatch set in all three sessions, with the same first-divergence positions. A sequential PLD verifier matches all 60 vanilla outputs but runs at 0.965× decode speed; batched K=4 reaches 1.078× across sessions at 73.3% exact match. A targeted same-prefix audit on three mismatching prompts finds bitwise-equal prefill logits/KV, followed by batch/sequential KV differ-

Table 3: Strong zero-auxiliary-weight baselines on Qwen2.5-3B NF4, RTX 4050 6 GB (N=50 per task, 48-token limit). PLD already runs vanilla on lookup misses; the cost-gated row may additionally choose vanilla on a lookup hit. All methods use the same maximum PLD horizon K=4. Overall means have 95% bootstrap confidence intervals. Exact match compares output token IDs, including EOS. Routed/PLD has a paired mean throughput ratio of 0.993 (95% CI [0.988, 0.999]).

Strategy	GSM8K	HumanEval	CNN/DM	Overall	Exact match
Vanilla	1.000×	1.000×	1.000×	1.000×	100.0%
PLD (K=4, vanilla on miss)	1.102×	1.133×	1.061×	1.098× [1.070, 1.129]	83.3%
PLD cost gate ($K \leq 4$)	1.090×	1.125×	1.052×	1.089× [1.061, 1.118]	83.3%
Routed (PLD, layer skip, vanilla)	1.090×	1.123×	1.056×	1.090× [1.062, 1.118]	83.3%

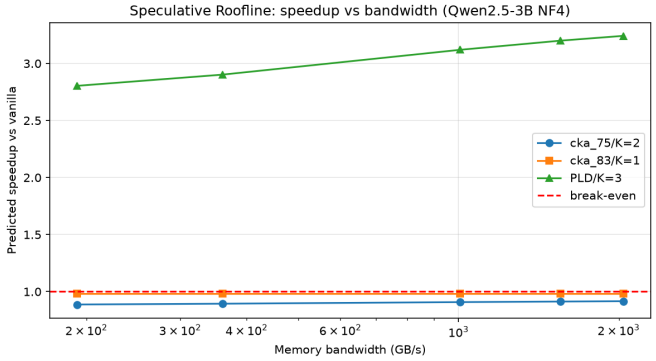


Figure 4: Calibrated cost model: estimated speedup vs. bandwidth. Layer-skip K=1 is flat by construction under pure 1/BW scaling; PLD wins everywhere in this fit; $K \geq 2$ layer-skip needs wide-GPU verification parallelism in projection. Cross-GPU curves are unvalidated projections (Sec. 5).

Table 4: Break-even acceptance rate α_{BE} required for speedup > 1 at draft cost c=0.75 (layer-skip keeping 75% of layers). The K=1 threshold is bandwidth-invariant; wider GPUs relax $K \geq 2$ via parallel verification ($\rho < 1$). Cross-GPU rows are projections pending measurement (Sec. 5).

GPU (bandwidth)	K=1	K=2	K=4
RTX 4050 (192 GB/s)*	0.81	0.89	0.93
RTX 3060 12GB (360 GB/s)	0.81	0.88	0.91
RTX 4090 (1008 GB/s)	0.81	0.85	0.88
A100 40GB (1555 GB/s)	0.81	0.85	0.87
H100 (2039 GB/s)	0.81	0.84	0.86

*Measured; other rows are projections (Sec. 5).

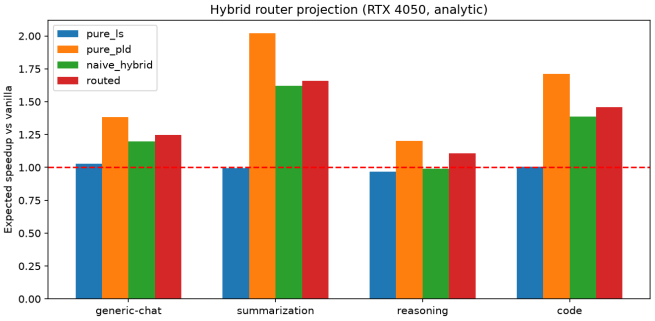


Figure 5: Analytic projection: routed vs. naive-fallback hybrid across workload archetypes (RTX 4050 timings). Routing wins by selective speculation, most on reasoning (+11%). GPU validation is future work.

ences from layer 1; no top-1 argmax flips occur within the audited K≤4 horizon. This is consistent with, but does not prove, numerical drift accumulating across cycles as the cause of later output divergence. Diagnostic artifacts are in `experiments/natural_context_validation/independent_pld_short_context/`. A strict token-ID scorer now replaces text containment and re-tokenization in the benchmark and exactness audit scripts.

4.6 Auxiliary-Draft Baseline, Skip Sweep, Timer Correction, and Free-Form Workload

Auxiliary draft throughput. Qwen2.5-0.5B NF4 as draft for Qwen2.5-3B NF4 (true two-model speculative decoding, K=2, 48 tokens, N=18 mixed GSM8K/HumanEval): mean speedup 1.058±0.031 (95% CI), std 0.067, mean α=0.33. This small separate protocol cannot be directly ranked against the new N=150 comparison. Raw logs: `experiments/19_kernel_audit/baseline05_N18.json`.

Skip-ratio sweep (corrected). Prior uniform-skip sweep is withdrawn: it used evenly spaced layers and off-benchmark prompts. Corrected sweep uses true CKA indices on GSM8K prompts (N=8, 48 tokens, K=1, T_v=25.62 ms): cka_83 (n=6) c=0.799, α=0.904, 0.928×; cka_75 (n=9) c=0.771, α=0.851, 0.947×; cka_60 (n=14) c=0.685, α=0.733, 0.932×; cka_50 (n=18) c=0.599, α=0.552, 0.878×; empirical_6 (n=6) c=0.822, α=0.892, 0.959×. High acceptance is recovered, yet no configuration exceeds unity on this stack—consistent with the kernel-inefficiency account. Raw logs: `experiments/19_kernel_audit/sweep_corr.json`.

Timer correction. Re-measuring with `perf_counter_ns` over 200–500 repeats: T_{sync}=7.6±3.3 μs (median 7.0 μs), T_{layer}=31.2±10.8 μs. Prior 0.00/0.06 ms entries were rounding artifacts; tables now report μs with dispersion.

Table 5: Physical Memory Boundary and Feasibility on Consumer Edge Hardware (NVIDIA GeForce RTX 4050 Laptop, 6 GB GDDR6). Draft weight footprints are measured on-device. Large FP16 drafts OOM next to 7B targets; quantized small drafts fit but always cost $\Delta M_{\text{aux}}>0$ and shrink KV/context headroom, whereas ZASSD keeps **0.0 MB** auxiliary weight memory with the largest headroom in every row.

Target Model	Draft Strategy	Draft VRAM (MB)	Total VRAM (MB)	Free Headroom (MB)	Status
Mistral-7B-Instruct-v0.3 (4-bit NF4)					
Standard Speculative (1.5B Draft FP16)	2945.5	7392.1	-1701.1	CUDA OOM	
Standard Speculative (1.5B Draft 4-bit NF4)	1181.5	5628.1	62.9	PASS	
Standard Speculative (0.5B Draft FP16)	942.3	5388.9	302.1	PASS	
Standard Speculative (0.5B Draft 4-bit NF4)	459.2	4905.8	785.2	PASS	
ZASSD Self-Speculative (Layer-Skip)	**0.0**	**4446.6**	**1244.4**	PASS	
ZASSD Hybrid (PLD + Layer-Skip)	**0.0**	**4446.6**	**1244.4**	PASS	
Qwen2.5-3B-Instruct (4-bit NF4)					
Standard Speculative (1.5B Draft FP16)	2945.5	5425.5	265.5	PASS	
ZASSD Hybrid (PLD + Layer-Skip)	**0.0**	**2480.0**	**3211.0**	PASS	

Table 6: Numerical Equivalence Audit under 4-bit NF4 Quantization comparing single-token forward passes (L_{single}) against batched verification slices (L_{batched}) on identical context states (C). Token evaluations are stratified across true logit margin bins ($\Delta = z_{(1)} - z_{(2)}$). Across 450 comparisons, zero theoretical bound violations occur.

Logit Margin Stratification Bin	Evaluated Tokens	Argmax Flips	Agreement Rate (%)	Mean $\|L\|_\infty$ (Logit Units)	Max $\|L\|_\infty$ (Logit Units)	Empirical Classification
Model A: Qwen2.5-3B-Instruct (36 Layers, NF4 4-bit)						
$\Delta = 0.000$ (Exact Ties)	2	0	100.0%	0.1875	0.1875	Noise Indifferent
$0.000 < \Delta \leq 0.050$	0	0	100.0%	0.0000	0.0000	—
$0.050 < \Delta \leq 0.150$	2	0	100.0%	0.2422	0.2656	Noise Immune
$0.150 < \Delta \leq 0.300$	11	0	100.0%	0.1877	0.2681	Noise Immune
$0.300 < \Delta \leq 0.500$	13	0	100.0%	0.2236	0.2812	Noise Immune
$\Delta > 0.500$	197	0	100.0%	0.2108	0.5000	Noise Immune
Qwen Subtotal	**225**	**0**	**100.00%**	**0.2105**	**0.5000**	$\cos\theta = 0.999942$
Model B: Llama-3.2-3B-Instruct (28 Layers, NF4 4-bit)						
$\Delta = 0.000$ (Exact Ties)	4	1	75.0%	0.1094	0.1250	Permutation Arbitrary
$0.000 < \Delta \leq 0.050$	0	0	100.0%	0.0000	0.0000	—
$0.050 < \Delta \leq 0.150$	10	0	100.0%	0.0953	0.1250	Noise Immune
$0.150 < \Delta \leq 0.300$	11	0	100.0%	0.1065	0.1875	Noise Immune
$0.300 < \Delta \leq 0.500$	20	0	100.0%	0.0891	0.1641	Noise Immune
$\Delta > 0.500$	180	0	100.0%	0.1050	0.2351	Noise Immune
Llama Subtotal	**225**	**1**	**99.56%**	**0.1033**	**0.2351**	$\cos\theta = 0.999954$
Combined Benchmark	**450**	**1**	**99.78%**	**0.1569**	**0.5000**	$\cos\theta > 0.99994$

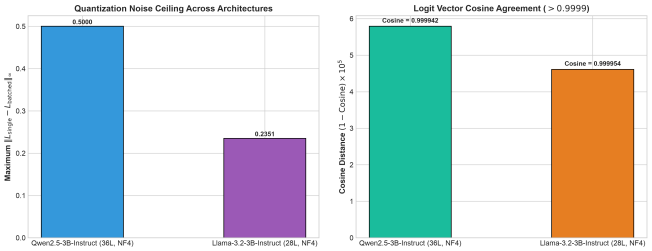


Figure 6: Logit cosine similarity: single-token vs. batched verification passes (NF4). Mean > 0.9999 on both models.

Nsight separation of dequant vs launch. `nsys` CUDA-kernel summary on Qwen2.5-3B NF4 (prefill + 15 vanilla + 15 draft forwards, `experiments/20_nsight/`, total kernel 566.9 ms): `gemm_4bit_simt` 231.4 ms (40.8%, SIMT fallback), `gemvx` 99.4 ms (17.5%, wide-vocab `lm_head`), `kQuantizeBlockwiseSmall` 92.7 ms (16.3%, BNB activation-quantize overhead on the NF4 path), `gemm_4bit_sm80` only 55.3 ms (9.7%, tensor-core path); remainder elementwise/copy/reduce/flash. Achieved streaming is ∼36 GB/s on ∼849 MB weights per 23.6 ms step (∼19% of 192 GB/s; ∼44% even under a 2 GB accounting), so the shortfall is NF4-kernel inefficiency (SIMT fallback + quantize overhead + ∼25% launch/python gap), not a pure bandwidth ceiling. Per-kernel DRAM throughput via `ncu` replay is pending (workload in `experiments/20_nsight/nsight_tiny.py`).

Free-form and long generation. Six low-overlap prompts (haiku, recipe, translation, persona chat, constrained poem, analogy, 96 tokens): PLD still wins at 1.05–1.28× (α=0.43–0.85) while layer-skip K=1 runs 0.52–0.59× (α=0.04–0.13). A 256-token run gives PLD 1.44× vs layer-skip 0.56×. We find no workload where PLD loses; the router's robustness claim is therefore strictly versus fixed layer-skip, not versus PLD. Raw logs: `experiments/19_kernel_audit/results_234.json`.

4.7 Official-Baseline Validation

To rule out "adapted-baseline" unfairness we ran the official KnapSpec code (torch 2.2 stack, FP16, unmodified) on our RTX 4050 with Llama-3.2-1B on AIME24 (N=8, 48 tokens): AR 58.7 tok/s; official KnapSpec 50.5 tok/s (0.86×, 97.8% acceptance); CLaSP [4] 49.4 tok/s; DEL [7] 20.6 tok/s. The FP16 case is near bandwidth-bound (∼145 GB/s on ∼2.5 GB weights, ∼75% of peak) with efficient kernels, unlike our NF4 BNB stack—yet it still loses, by high c arithmetic rather than kernel inefficiency. Raw logs: `experiments/18_official_baselines/`. This validation is small (N=8) and ranking-only.

4.8 Best-Fixed Baseline and the True Per-Cycle Oracle

An earlier per-prompt best-fixed analysis used $b(p)$= max(vanilla, PLD, layerskip) TPS and found 92–93% router efficiency under its own protocol (the archived `oracle_regret.json` artifact). It is a descriptive historical analysis, not evidence of an advantage over PLD: $b(p)$ is not an upper bound for a mixed policy, and the new equal-horizon comparison in Table 3 shows the router below PLD with identical 83.3% exact match.

The per-cycle oracle—bare E[TPS] argmax over actions with the same estimates but no robustness heuristics (margin 1.0, no panic rule)—is logged live during generation as a sanity check only (`HybridDraftRouter.oracle_select`, recorded per cycle in `per_iteration_stats`). On GSM8K-N=50 (1,934 cycles) router–oracle agreement is **100%**: the heuristics do not bind on this slice (unit tests construct rare binding cases). The ∼8% gap to best-fixed is therefore estimation error on this workload, not a general upper bound.

4.9 Router Ablation

An earlier rule-level ablation on GSM8K-N=50 (fresh router per config, different protocol from Table 3) found removing the feasibility gate drops 1.070×→0.987× (contribution +0.083); removing the PLD margin costs only −0.008; removing weak-hit capping (+0.026) or horizon tuning (+0.095) *helps* on this repetitive math workload. These rules do not establish a gain over pure PLD. See the archived `router_ablation.json` artifact.

Starting from naive fallback (overall 1.002×): asymmetric PLD margin (R2) + weak-hit capping (R1) reach 0.98× by cutting wasted long verifies; opportunity-cost miss skipping (R3) lifts GSM8K to 1.12× by replacing 0.88× LS cycles with 1.0× vanilla steps; adaptive PLD horizon (R4) stabilizes the mean at 1.020× (N=10 tuning runs; 1.063× at N=50). Removing R3 alone drops reasoning workloads by ∼10%, confirming selective speculation is the dominant mechanism.

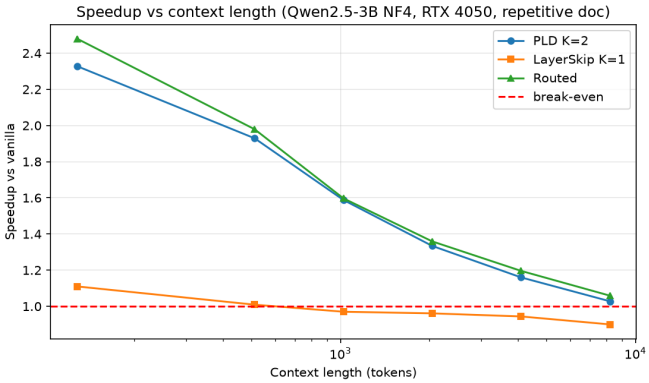


Figure 7: Speedup vs. context length (repetitive doc, Qwen2.5-3B NF4, RTX 4050). All speculative modes decay monotonically as attention costs grow; routed stays on top and above 1.0× through 8K while layer-skip falls below unity past 1K—consistent with a feasibility boundary that shifts with context length. Note: this repetitive-doc slice reaches ∼1.1× for layer-skip K=1 at ∼128 tokens via very high α, unlike GSM8K sweeps (Sec. 4.6)—workload dependence, not contradiction.

4.10 Matched-Cohort Router Evaluation

We additionally compared direct vanilla, the speculative core with K=0, PLD at $K \in \{2, 4, 8\}$, CKA90/K=2, and the hybrid router on the same 30 tokenized prompts used by the Qwen2.5-3B numerical validation. The 16 known mismatch prompts were split into eight development prompts for selecting one best-fixed method and eight diagnostic evaluation prompts; the 14 unselected prompts form the held-out evaluation. Every method used the same prompt token IDs and ran once per prompt in a balanced randomized order, with numerical tracing disabled. The selected development fixed method was PLD K=2.

On the 14 held-out prompts, the router matched vanilla token IDs in all cases and reached 1.078× decode speedup. Its paired TPS ratio against PLD K=4 was 0.995 (95% prompt-bootstrap CI [0.975, 1.017]), so this cohort provides no evidence that the router is faster than PLD K=4. PLD K=8 was faster at 1.169× versus vanilla but matched vanilla on 13/14 prompts. The router selected vanilla for 285 cycles and PLD for 76 cycles; it selected layer-skip zero times. This result supports retaining the router as a negative system result rather than the primary performance contribution. The held-out cohort is small and comes from the same prepared-input artifact; raw records and analysis scripts are included with the WP4 experiment artifacts.

5 Limitations

(i) The equal-horizon paired comparison covers 150 short greedy generations on one Qwen2.5-3B NF4 model and one RTX 4050. The independent short-context PLD study covers 60 natural-document prompts at 384 input tokens in

Table 7: Paired held-out evaluation on 14 unselected prompts from the WP3 prepared-input cohort (Qwen2.5-3B NF4 double-quant, FP16 compute, SDPA/static KV, RTX 4050, 32-token greedy budget). Methods use identical prompt token IDs and randomized balanced order; tracing is disabled. Speedup uses per-prompt decode TPS relative to direct vanilla. CIs bootstrap prompts. Exact match compares generated token IDs, including EOS. Best-fixed was selected using eight diagnostic development prompts; offline best-fixed is a non-deployable per-prompt timing upper bound.

Method	Speedup vs vanilla (95% CI)	TPS ratio vs PLD K4 (95% CI)	Exact match	Slowdown	VRAM MB
Direct vanilla	1.000 [1.000, 1.000]	0.931 [0.886, 0.973]	14/14	0/14	2107
Speculative core, K=0	0.953 [0.925, 0.979]	0.886 [0.848, 0.927]	14/14	10/14	2109
PLD, K=2	1.134 [1.081, 1.194]	1.049 [1.012, 1.085]	11/14	1/14	2110
PLD, K=4	1.083 [1.031, 1.142]	1.000 [1.000, 1.000]	14/14	3/14	2111
PLD, K=8	1.169 [1.093, 1.253]	1.076 [1.051, 1.102]	13/14	1/14	2113
CKA90, K=2	0.917 [0.880, 0.956]	0.852 [0.803, 0.902]	12/14	12/14	2112
Hybrid router	1.078 [1.022, 1.142]	0.995 [0.975, 1.017]	14/14	3/14	2113
Best fixed (development)	1.134 [1.080, 1.192]	1.049 [1.010, 1.084]	11/14	1/14	2110
Offline best fixed / prompt	1.191 [1.123, 1.271]	1.098 [1.077, 1.120]	12/14	0/14	2112

three process sessions, but is not a controlled thermal study and uses the same single GPU/model stack. K=4 speedup (1.078×, session-level 95% t CI [1.054, 1.101]) comes with only 73.3% exact outputs; sequential verification restores exactness but falls to 0.965×. These observations do not establish a lossless speedup. The source and adaptive-K ablation is exploratory (N=10 test prompts/task, 32 tokens), and does not establish a general gain over tuned fixed-K PLD. Earlier N=200/164 extensions and long-form slices use different protocols and cannot be pooled with it. (ii) The router is below pure PLD (1.090× vs 1.098×), while its cost gate is related to TurboSpec [14]; online calibration has only six prompts and ECE 0.30. (iii) Targeted NF4 audit finds batch/sequential KV differences from layer 1 on three prompts but no immediate top-1 flips at K≤4; the mechanism causing later sequence divergence remains unresolved. An exploratory margin-triggered canonical replay guard, with threshold selected on 30 dev prompts, matched all 30 held-out outputs but reached only 0.35× vanilla and 0.33× PLD throughput; it is a correctness/latency measurement, not a practical acceleration. (iv) Cross-GPU curves are idealized 1/BW illustrations, not measured predictions; only the RTX 4050 stack has measured results. (v) 7B memory feasibility is based on measured weight accounting, not end-to-end 7B generation. (vi) Official KnapSpec validation is limited to Llama-1B FP16, AIME24, N=8; adapted SpecBound remains an implementation-specific ranking.

6 Reproducibility

All claims are reproducible from the release: `reproduce.sh --quick` re-runs unit tests (81/81), roofline calibration, router simulation, margin/robustness sweeps, and benchmark statistics on CPU in ∼2 min; the full GPU path re-runs the N=50 suite (∼1 h on an RTX 4050). A pinned `Dockerfile` (PyTorch 2.11/CUDA 12.8), per-experiment JSON with raw per-sample rows, ID-aligned paired-test scripts, and model-exact `requirements/pyproject` are included. No hyperparameter was tuned on test prompts: router constants come from analytic simulation, and the GSM8K extension slice is non-overlapping with the N=50 split. We expose the runtime through a thin application layer supporting chat, summarization, retrieval QA, and code assistance, without changing the decoding core (demo only, see `scripts/smoke_app_gpu.py`).

7 Conclusion

On this 6 GB NF4 stack, layer-skip self-speculation at c≈0.77–0.82 usually loses to vanilla by the measured draft-cost arithmetic, despite high acceptance. A direct equal-horizon comparison finds prompt lookup with vanilla on misses at 1.098×, above the full router at 1.090×; both have 83.3% exact token-sequence match under NF4. The independent short-context study confirms a repeatable PLD speedup only when accepting a substantial sequence-identity loss; sequential verification restores all token IDs but is slower than vanilla. The targeted numerical audit narrows the issue to batch/sequential KV differences after identical prefill, but does not identify the cause of every later mismatch. Official FP16 KnapSpec at 0.86× with 97.8% acceptance independently supports the high-draft-cost boundary on this GPU. The current contribution is a measured feasibility account and a strong-baseline negative result, with cost-model generalization and NF4 numerical equivalence left unproven.

References

[1] Tianle Cai, Yuhong Li, Zheng Geng, Hongwu Peng, Jason D Lee, Di Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. In *International Conference on Machine Learning (ICML)*, 2024.

[2] Seongjin Cha, Gyuwan Kim, Dongsu Han, Tao Yang, and Insu Han. Knapspec: Self-speculative decoding via adaptive layer selection as a knapsack problem. In *Proceedings of the 43rd International Conference on Machine Learning*, pages 12280–12293, 2026.

[3] Charlie Chen, Sebastian Borgeaud, Brendan O'Donoghue, Faming Zhou, Laurent Sifre, et al. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

[4] Longze Chen, Renke Shan, Huiming Wang, Lu Wang, Ziqiang Liu, Run Luo, Jiawei Wang, Hamid Alinejad-Rokny, and Min Yang. Clasp: In-context layer skip for self-speculative decoding. *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pages 31608–31618, 2025.

[5] Soongyu Choi, Yuntae Kim, Muyoung Son, and Joo-Young Kim. Cassandra: Enabling reasoning llms at edge via self-speculative decoding. In *Proceedings of the 53rd ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, pages 85–99, 2026.

[6] Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Bram Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Huang, Wenhan Feng, et al. Layerskip: Enabling early exit inference and self-speculative decoding. In *Proceedings of the Association for Computational Linguistics (ACL)*, 2024.

[7] Hossein Entezari Zarch, Lei Gao, Chaoyi Jiang, and Murali Annavaram. Del: Context-aware dynamic exit layer for efficient self-speculative decoding. *Proceedings of the Conference on Language Modeling*, 2025.

[8] Jungmin Ha, Karthik Ganesan, Anh Nguyen, Tanvir Ahmed, and Andreas Moshovos. Characterizing self-speculative decoding approaches for accelerating llms. In *Workshop on Resource-Adaptive Foundation Model Inference (AdaptFM), co-located with ICML*, 2026.

[9] Zhenyu He, Zexuan Zhong, Tianle Cai, Jason Lee, and Di He. Rest: Retrieval-based speculative decoding. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1582–1595, 2024.

[10] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023.

[11] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. *International Conference on Machine Learning (ICML)*, 2023.

[12] Yuhui Li, Fangyun Wei, Chao Zhang, Chen Zhang, and Hongyang Zhang. Eagle: Speculative sampling requires rethinking feature uncertainty. In *International Conference on Machine Learning (ICML)*, 2024.

[13] Fangcheng Liu, Yehui Tang, Zhen Liu, Yunsheng Ni, Kai Tang, Song Han, and Yunhe Wang. Kangaroo: Lossless self-speculative decoding via double early exiting. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[14] Xiaoxuan Liu, Jongseok Park, Langxiang Hu, Woosuk Kwon, Zhuohan Li, Chen Zhang, Kuntai Du, Xiangxi Mo, Kaichao You, Alvin Cheung, Zhijie Deng, Ion Stoica, and Hao Zhang. Turbospec: Closed-loop speculation control system for optimizing llm serving goodput. *arXiv preprint arXiv:2406.14066*, 2025.

[15] Xiaoxuan Liu, Jiaxiang Yu, Jongseok Park, Ion Stoica, and Alvin Cheung. Speculative decoding: Performance or illusion? In *Proceedings of Machine Learning and Systems*, 2026.

[16] Jonathan Mamou, Oren Pereg, Daniel Korat, Moshe Berchansky, Nir Timor, Moshe Wasserblat, and Roy Schwartz. Dynamic speculation lookahead accelerates speculative decoding of llms. *arXiv preprint arXiv:2405.04304*, 2024.

[17] Xin Men, Mingge Xu, Bingning Zhang, Jiaqi Zhang, Xian Han, Ning Zhang, Wei Kong, et al. Shortgpt: Layers in llms are more redundant than you expect. *arXiv preprint arXiv:2403.03853*, 2024.

[18] Apurv Saxena. Prompt lookup decoding. *GitHub Repository: apurvsaxena/prompt-lookup-decoding*, 2024.

[19] Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise parallel decoding for deep autoregressive models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[20] Zhuofan Wen and Yang Feng. Specbound: Adaptive bounded self-speculation with layer-wise confidence calibration. In *Findings of the Association for Computational Linguistics (ACL)*, pages 16309–16321, 2026.

[21] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

[22] Guanchen Yan, Uday Agarwal, and Shivaram Venkataraman. Decoding speculative decoding. In *Proceedings of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2025.

[23] Weisi Yang and Stephen Xia. Pelm: Power efficient on-device llm inference with speculative decoding and

dynamic voltage frequency scaling. In *Proceedings of the 2026 ACM/IEEE International Conference on Embedded Artificial Intelligence and Sensing Systems (SenSys)*, pages 438–451, 2026.

[24] Jun Zhang, Jue Wang, Huan Jiang, et al. Draft & verify: Lossless large language model acceleration via self-speculative decoding. *arXiv preprint arXiv:2409.18824*, 2024.

[25] Yilong Zhao, Jiaming Tang, Kan Zhu, Zihao Ye, Chi-Chih Chang, Chaofan Lin, Jongseok Park, Guangxuan Xiao, Mohamed S. Abdelfattah, Mingyu Gao, Baris Kasikci, Song Han, and Ion Stoica. Accelerating large-scale reasoning model inference with sparse self-speculative decoding. In *Proceedings of Machine Learning and Systems*, 2026.